Building a Limit Order Book in C++: A Complete Tutorial

Nov 2, 2026 · @alison

Overview

You will build a working limit order book in C++17, a matching engine around it, four kinds of simulated traders, and a deterministic simulator that makes them trade with each other, then a faster second book that is measured against the first.

The tutorial assumes you code well, know data structures and systems, and understand basic finance, but are new to C++ syntax. Each code block is followed by what it does, an explanation of any C++ construct you may not have met, and a **port it** note: the idea stripped of C++ so you can rebuild it in Rust, Go, Java or Python.

Part	File	What it does
1	`types.hpp`	Price, quantity, order, trade and event types
2	`order_book.hpp`	One-symbol book: price-time priority, add, cancel, modify, match
3	`engine.hpp`	Order entry: validation, ids, timestamps, order types, self-trade prevention, event feed
4	`agents.hpp`	Market maker, noise trader, momentum trader, informed trader
5	`sim.hpp`, `main.cpp`	Discrete-event simulator with latency, a hidden fundamental value and CSV output
6	`tests.cpp`, `bench.cpp`	Unit tests and a benchmark harness
7	`fast_book.hpp`	Flat-array book with a pooled order store, measured against Part 2

All files live in one folder and are compiled together with a single command:

```
g++ -std=c++17 -O2 -Wall -Wextra -o sim main.cpp
./sim
```

Two conventions run through everything. Prices are integers counted in ticks (never floating point), and time is a simulated integer clock in nanoseconds. Both choices are explained where they first matter.

Read Parts 1 to 3 for the book itself, Parts 4 and 5 to bring it to life, then the sections after Part 7 for flaws, improvements, optimisation, exploits and applications.

C++ crash course

About a dozen C++ ideas are enough to read every file in this tutorial. This section covers only those, each mapped to something you already know from another language.

Compiling

`#include "types.hpp"` pastes that file's text into yours before compilation, and `#pragma once` stops the same header being pasted twice. Everything here lives in headers, so `g++ main.cpp` builds one translation unit: one compiler run over one expanded source file. Larger projects split declarations (`.hpp`) from definitions (`.cpp`) to speed up rebuilds, and the logic is unchanged.

The flags: `-std=c++17` picks the language version, `-O2` turns on optimisation, `-Wall -Wextra` turns on useful warnings. Treat every warning as a bug.

Values, references and pointers

A C++ variable of a class type holds the object itself, not a reference to a heap object as in Java or Python. Assigning or passing it copies it unless you ask otherwise.

```
void a(Order o);          // gets its own copy; changes stay local
void b(const Order& o);   // refers to the caller's object; read-only; no
copy
void c(Order& o);         // refers to the caller's object; may modify it
void d(const Order* o);   // nullable address; check for nullptr before use
```

`OrderBook::submit(Order o, ...)` takes a copy on purpose: it stamps the arrival time and shrinks `qty` while matching, and the caller's original must stay untouched. `match(Order& in, ...)` takes a reference because it must shrink that working copy in place.

Classes

- `struct` and `class` are identical except that members default to public in a `struct` and private in a `class`.

- A constructor's initialiser list (`: listener_(listener), stp_(flag) {}`) sets members before the body runs. Members initialise in declaration order, not list order, which is why `Engine` declares `book_` first.
- `virtual` marks a method that subclasses may replace, and `override` makes the compiler check that you really are replacing one. A class with virtual methods needs a `virtual` destructor so deleting through a base pointer is safe. `= default` asks the compiler to write the obvious version.
- `explicit` on a one-argument constructor blocks accidental implicit conversions.
- A trailing `const` (`best_bid() const`) promises the method does not modify the object.

Containers and iterators

C++ type	Closest equivalent	What we rely on
`std::map`	Java `TreeMap`, Python `sortedcontainers`	Sorted iteration, O(log n) operations
`std::unordered_map`	Hash map	O(1) average lookup; iterators die on rehash, element addresses do not
`std::list`	Doubly linked list	O(1) erase by iterator; other elements' iterators stay valid
`std::vector`	Dynamic array	Contiguous memory; `push_back` may reallocate and invalidate every reference
`std::deque`	Double-ended queue	O(1) at both ends
`std::optional<T>`	Nullable value, Rust `Option`	Says "maybe no value" in the type

An **iterator** is a pointer-like handle into a container: `*it` is the element, `it->first` reads a member, `++it` advances. `map.begin()` is the first key in sort order, which is why the best price is always `begin()`.

Templates

A template is a function or class with type placeholders that the compiler fills in at build time. It behaves like Rust generics rather than Java generics: a separate copy is compiled for each type. `match` is written once as `template <class Book> void match(Order&, Book&, Time)` and serves both sides of the book. Bids and asks are different types

because the sort comparator is part of a map's type, so ordinary overloading could not share the code. A lambda with an `auto` parameter, `[&](const auto& book) { ... }`, is the same idea for a local function.

Lambdas and std::function

`[captures](params) { body }` is an anonymous function. The capture list says what it can see: `[&]` captures locals by reference, `[this, id]` captures the enclosing object plus a copy of `id`. A reference to something that dies before the lambda runs is undefined behaviour, so lambdas stored for later (as the simulator does) capture by value. `std::function<void()>` can hold any callable of that shape, like Java's `Runnable`.

Modern conveniences

```
for (const auto& [price, level] : book) { /* ... */ }  // structured binding:
unpack a pair
auto it = index_.find(id);                              // auto: compiler
infers the type
if (it == index_.end()) return false;                   // find() returns
end() when absent
std::optional<Price> p = best_bid();                    // check before use
if (p) use(*p);
```

Ownership

There is no garbage collector. A local object dies at the end of its scope and its destructor runs; this pattern is called RAII. Heap ownership is expressed with `std::unique_ptr<T>`, a pointer that deletes its object when it goes out of scope. It cannot be copied, only moved with `std::move`. `std::make_unique<T>(args...)` allocates and constructs in one step, and the simulator owns its agents this way.

Four traps

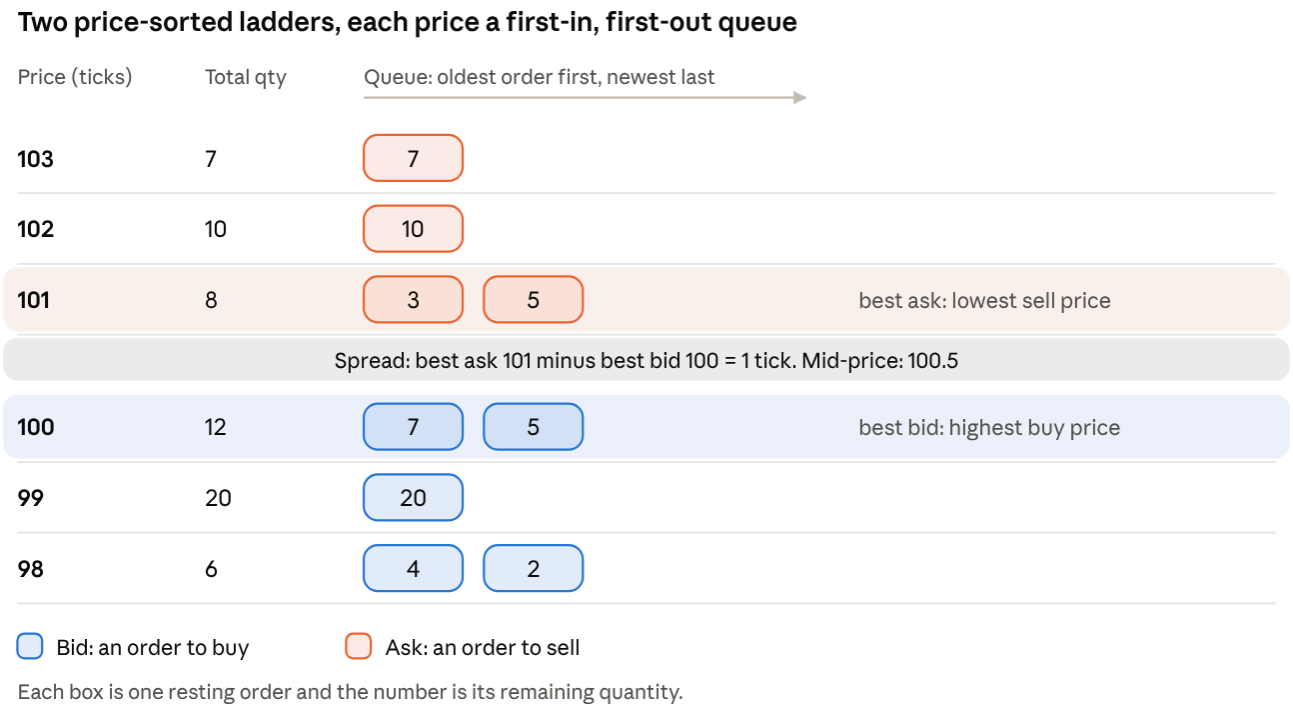
1. **`map[key]` inserts.** Reading `m[k]` for a missing key creates a default element. `rest()` uses that on purpose, but use `find()` when you only want to look.
2. **Dead iterators and references.** After `vector::push_back` reallocates, or after the element a handle points to is erased, the handle is invalid. Using it is undefined behaviour, and it can appear to work for months.
3. **Uninitialised scalars** hold garbage, so every member here has a default (`Qty qty = 0;`).

4. **Integer pitfalls.** Signed overflow is undefined, division truncates toward zero, and mixing signed with unsigned converts silently (`-1 < 1u` is false). The project uses `int64_t` throughout to avoid most of this.

Anatomy of an order book

An order book is two sorted collections of resting limit orders, bids to buy and asks to sell, plus one rule: whenever the best bid meets or exceeds the best ask, trade, highest price first and oldest order first within a price.

The ladder



illustrative book · 6 price levels, 9 resting orders

Prices are integers, and each price level holds a queue in arrival order whose front trades first. The best bid and best ask, together called the top of book or the touch, sit on either side of the spread. Nothing is ever stored between them: if a bid and an ask could trade, they already have.

Vocabulary

Term	Meaning	In our code
Bid / ask	An order to buy / to sell at a limit price	`Side::Buy` / `Side::Sell`

Term	Meaning	In our code
Best bid, best ask	Highest resting buy, lowest resting sell	`best_bid()`, `best_ask()`
Spread	Best ask minus best bid; the cost of crossing immediately	Derived from `Engine::Quote`
Mid-price	Midpoint of best bid and best ask	Computed by the agents
Tick size	Smallest price increment; every price is a whole number of ticks	`Price` counts ticks
Level	All resting orders at one price	`PriceLevel`
Maker / taker	The resting order / the incoming order that trades against it	`Trade::maker_id`, `taker_id`
Depth	Quantity available at each level away from the touch	`depth()`
Marketable	An order that can trade the moment it arrives	`crosses()`

Order types

Type	Behaviour	Rests if unfilled?	Typical use
Limit (GTC)	Trade at the limit price or better, rest the remainder	Yes	Providing liquidity, patient entry
Market	Trade at any price until filled or the book is empty	Never	Urgency, with an unpredictable price
IOC (immediate-or-cancel)	Limit order: trade what you can now, cancel the rest	No	Controlled aggression
FOK (fill-or-kill)	Trade the whole quantity now, or nothing	No	All-or-nothing execution
Post-only	Limit order rejected if it would trade immediately	Yes	Guaranteeing maker status (and maker fee rebates)

The code implements all five. Not implemented: stop and stop-limit orders (a trigger that releases an order when the price moves), iceberg orders (displayed size smaller than true size) and auctions.

Matching rules

Price-time priority, also called FIFO, is the default on most equity and futures venues.

1. An incoming order is **marketable** if its limit reaches the best opposite price. A market order always is.
2. It trades against the best opposite level first, and within that level against the oldest order first.
3. Each trade happens at the **resting** order's price, so a buyer willing to pay 102 who meets an ask at 101 pays 101.
4. If quantity remains, a GTC limit order rests at its own limit, at the back of that level's queue. Anything else is cancelled.

Other allocation rules exist. Pro-rata splits a fill across the queue by size and is used in some interest-rate futures. Size priority favours larger orders. Auctions collect orders and cross them at a single price, as in the opening and closing crosses.

Worked example

The asks are 101 → [3, 5], 102 → [10], 103 → [7], with resting bids below 101. A buy limit order for 20 at 102 arrives.

Step	Resting order hit	Price	Fill	Incoming remaining
1	First order at 101	101	3	17
2	Second order at 101	101	5	12
3	The order at 102	102	10	2
4	Next ask is 103, above the limit of 102, so stop	none	0	2

The remaining 2 rest as a bid at 102, which becomes the best bid, and the best ask is now 103. The buyer paid 101 for 8 units and 102 for 10, an average of about 101.56, better than the 102 limit.

Market data levels

Level 1 (L1) is the best bid and ask with their sizes. Level 2 (L2) is the aggregated quantity at each price down to some depth, which is what `depth()` returns. Level 3 (L3) lists every individual order with its id, as order-by-order exchange feeds such as Nasdaq ITCH do.

Invariants

A correct book keeps these true after every operation. `check_invariants()` verifies the first four, and the random tests call it thousands of times.

1. The book is never crossed: best bid < best ask.
2. No empty price level exists.
3. Each level's cached total equals the sum of its orders.
4. Every resting order can be found by id, and the id points at that very order.
5. Within a level the queue order is arrival order. The priority tests check this through the order of trades.

Design decisions

The book is a sorted map of price levels, each holding a linked list of orders, plus a hash map from order id to the order's position. That combination makes every operation the book needs cheap, and it is the design most reference implementations start from.

What each operation needs

Operation	What it must do	Structure that serves it	Cost
Best bid / ask	Read the top price	Sorted map, `begin()`	O(1)
Add a resting order	Find or create its price level, append	Sorted map, then list `push_back`	O(log L)
Match	Walk the best level's queue from the front	List `front()`, `pop_front()`	O(1) per fill
Cancel	Remove an order anywhere in a queue	Hash lookup of a stored list iterator	O(1) average, plus O(log L) to find the level
Depth snapshot	First n levels in order	Sorted map iteration	O(n)

L is the number of distinct price levels, usually a few hundred. Each of those choices is explained below.

Why integers for price

Binary floating point cannot represent 0.1 exactly, so two prices that should be equal may differ in the last bit, and then they land in different levels or compare the wrong way. Real venues quote in a fixed tick size, so a price is a whole number of ticks. Here one tick is $0.01 and a price of 10000 means $100.00. Prices, quantities and cash are all `int64_t`, so a run of 60 simulated seconds cannot overflow.

Why a sorted map for levels

A `std::map` keeps keys sorted, so the best price is always the first element and the matching loop never has to search. The bid side uses `std::greater` so that its first element is the highest price. A red-black tree gives O(log L) inserts and erases. A heap would give the best price too, but cannot cheaply delete an arbitrary level or iterate in order for a depth snapshot.

Why a linked list per level

The queue needs three things: append at the back, remove from the front when matched, and remove from the middle when cancelled. A `std::list` does all three in O(1) once you hold an iterator, and its iterators stay valid when other elements are added or removed. That last property is what lets the id index store an iterator safely. A vector would make cancel O(n) in queue length, and a deque cannot delete from the middle cheaply.

Why an id index

A cancel request carries only an order id. Without an index you would scan every level. The `unordered_map<OrderId, Locator>` stores the side, price and list iterator, so a cancel is one hash lookup, one tree lookup to reach the level's cached total, and one list erase.

Why a cached total per level

Depth snapshots and fill-or-kill checks need the quantity at a level. Recomputing it walks the queue. The book keeps `total` in sync on every change, and `check_invariants()` proves it never drifts.

Other designs

Design	Idea	Better when	Worse when
Flat array per tick	Index the level directly by price offset	Price range is bounded and dense (this tutorial's `FastBook`)	Wide or sparse price range, such as crypto or tiny tick sizes
Intrusive linked list in a pool	Orders live in one array and link by index	You want no per-order allocation and good locality	You want simple code
Sorted vector of levels	Levels contiguous, best at the end	Few levels, mostly touching the top	Many inserts in the middle
Order-by-order feed handler	Maintain a book from exchange messages	Building a strategy rather than an exchange	Not applicable to a matching engine

The tutorial builds the first design (`std::map` plus `std::list`) because it is the easiest to get right, then implements the flat-array variant in the optimisation section and measures the difference.

Part 1: Core types (`types.hpp`)

Every other file uses this vocabulary, so it comes first. Nothing here has behaviour except a few small helpers.

```
#pragma once
#include <cstdint>

namespace ob {

using Price   = std::int64_t;  // integer number of ticks, never a float
using Qty     = std::int64_t;  // signed, so "position -= fill" is safe
using OrderId = std::uint64_t;
using AgentId = std::uint32_t;
using Time    = std::int64_t;  // simulated nanoseconds

constexpr Time US  = 1'000;
constexpr Time MS  = 1'000'000;
constexpr Time SEC = 1'000'000'000;

enum class Side : std::uint8_t { Buy, Sell };
enum class OrderType : std::uint8_t { Limit, Market };
```

```
enum class TimeInForce : std::uint8_t { GTC, IOC, FOK };

constexpr Side opposite(Side s) { return s == Side::Buy ? Side::Sell :
Side::Buy; }
```

What to notice:

- `using Price = std::int64_t;` is a type alias. It costs nothing at run time, and it means that changing the width of a price later is a one-line edit. It also documents intent at every use.
- `namespace ob { ... }` keeps the names out of the global scope, like a module or package.
- `enum class` is a scoped enumeration: you must write `Side::Buy`, and it does not silently convert to an integer. A plain `enum` does both, which is a classic source of bugs. `: std::uint8_t` fixes its storage at one byte.
- `constexpr` means the value or function is computed by the compiler. The digit separators in `1'000'000` are only for readability.

```
struct Order {
    OrderId     id        = 0;
    AgentId     owner     = 0;
    Side        side      = Side::Buy;
    OrderType   type      = OrderType::Limit;
    TimeInForce tif       = TimeInForce::GTC;
    bool        post_only = false;
    Price       price     = 0;  // ignored for market orders
    Qty         qty       = 0;  // remaining quantity; shrinks as the order
fills
    Time        ts        = 0;  // arrival time at the book (stamped by the
book)
};

struct Trade {
    OrderId taker_id;
    OrderId maker_id;
    AgentId taker_owner;
    AgentId maker_owner;
    Side    taker_side;  // the aggressor's side
    Price   price;       // always the resting (maker) order's price
    Qty     qty;
    Time    ts;
};
```

`= 0` after a member is a default member initialiser. C++ does not zero fields for you, so an unset `int` holds whatever bytes were in memory. `Trade` has no defaults because it is only ever built all at once, with the brace syntax `Trade{a, b, c, ...}`, which fills fields in declaration order.

`qty` is the *remaining* quantity, not the original. That is a deliberate choice: the book needs only what is left, and a fill is a subtraction. A real system also stores the original quantity for reporting.

The event interface

```
class Listener {
public:
    virtual ~Listener() = default;
    virtual void on_accept(const Order&, Time) {}
    virtual void on_reject(const Order&, RejectReason, Time) {}
    virtual void on_trade(const Trade&) {}
    virtual void on_cancel(const Order&, CancelReason, Time) {}
};
```

The book never prints, logs or calls an agent. It tells a `Listener` what happened and nothing more. This is the observer pattern, and it is the single most useful design decision in the project: the same book can feed a logger in the tests, the engine's statistics in production, and the simulator's agents, without changing a line of matching code. The empty default bodies mean a listener overrides only the events it cares about.

`virtual` makes the call go through a table of function pointers at run time, so the book calls the right subclass without knowing its type. The cost is one indirect call per event, which the optimisation section returns to.

Port it: an interface or trait with four callbacks, plus plain structs for the data. In Rust, a `trait Listener` with default methods; in Java, an interface with default methods; in Python, a base class whose methods do nothing.

Part 2: The OrderBook class (`order_book.hpp`)

This is the heart of the project: 272 lines, of which the matching loop is about thirty. It is read in four pieces: the data, the matching loop, resting and removing orders, and order entry.

The data

```
struct PriceLevel {
    std::list<Order> fifo;   // oldest first
    Qty              total = 0;
};
using BidMap = std::map<Price, PriceLevel, std::greater<Price>>;
using AskMap = std::map<Price, PriceLevel, std::less<Price>>;

struct Locator {
    Side                     side;
    Price                    price;
    std::list<Order>::iterator it;
};

BidMap bids_;
AskMap asks_;
std::unordered_map<OrderId, Locator> index_;
```

The third template argument of `std::map` is the comparator. `std::greater<Price>` sorts descending, so `bids_.begin()` is the highest bid, and `std::less<Price>` sorts ascending, so `asks_.begin()` is the lowest ask. Both sides therefore read "best price = `begin()`", and the matching code never has to ask which way is better. The cost is that `BidMap` and `AskMap` are two different types, which is why the functions that handle either side are templates.

A trailing underscore on a member name (`bids_`) is a convention for "private data", not syntax.

The matching loop

```
// `book` is the OPPOSITE side of the incoming order.
// begin() is always the best price on that side, because of the map
ordering.
template <class Book>
void match(Order& in, Book& book, Time now) {
    while (in.qty > 0 && !book.empty()) {
        auto level_it = book.begin();
        if (!crosses(in, level_it->first)) break;
        PriceLevel& level = level_it->second;

        while (in.qty > 0 && !level.fifo.empty()) {
            Order& maker = level.fifo.front();  // oldest order at this price

            if (stp_ && maker.owner == in.owner) {  // self-trade: cancel the
```

```
resting one
                const Order dead = maker;
                level.total -= dead.qty;
                index_.erase(dead.id);
                level.fifo.pop_front();
                if (listener_) listener_->on_cancel(dead,
CancelReason::SelfTrade, now);
                continue;
            }

            const Qty fill = std::min(in.qty, maker.qty);
            in.qty      -= fill;
            maker.qty   -= fill;
            level.total -= fill;
            if (listener_)
                listener_->on_trade(Trade{in.id, maker.id, in.owner,
maker.owner,
                                          in.side, level_it->first, fill,
now});
            if (maker.qty == 0) {
                index_.erase(maker.id);
                level.fifo.pop_front();
            }
        }
        if (level.fifo.empty()) book.erase(level_it);  // never keep empty
levels
    }
}
```

Read it as two nested loops. The outer loop walks price levels from best to worse and stops as soon as the next level no longer crosses the incoming order's limit. The inner loop walks one level's queue from oldest to newest. Each iteration of the inner loop either exhausts the incoming order or exhausts one resting order, so the loop always terminates.

C++ points:

- `Order& in` is a reference to the caller's working copy, so `in.qty -= fill` changes it in place. `PriceLevel& level` and `Order& maker` are references into the book itself, so `maker.qty -= fill` changes the resting order without a lookup.
- `level_it->first` is the price (the key) and `level_it->second` is the level (the value).
- `const Order dead = maker;` copies the order *before* `pop_front()` destroys it. Using `maker` after the pop would be undefined behaviour, and it often appears to work. The same care is taken before `index_.erase(maker.id)`: the id is read first, then the element is destroyed.

- `book.erase(level_it)` invalidates `level_it` and `level`. The loop then immediately re-reads `book.begin()`, never the old iterator.
- The trade price is `level_it->first`, the resting order's price, which is what makes the price-improvement rule hold.

Self-trade prevention (STP) is a policy choice. Here the *resting* order is cancelled and the incoming order carries on, which is the "cancel oldest" mode some exchanges offer. Other modes cancel the incoming order, both, or reduce the larger by the smaller. An exchange must pick one because a firm trading with itself creates wash trades, which distort volume and are illegal in most markets.

Port it: the matching loop is a `while` over the best level and a `while` over its queue. In any language, make sure you capture an order's fields before removing it from the queue, and delete a level the moment it is empty.

Resting and removing

```
template <class Book>
void rest(Book& book, const Order& o) {
    PriceLevel& level = book[o.price];  // operator[] creates the level if
missing
    level.fifo.push_back(o);
    level.total += o.qty;
    index_.emplace(o.id, Locator{o.side, o.price,
std::prev(level.fifo.end())});
}

std::optional<Order> remove_resting(OrderId id) {
    auto found = index_.find(id);
    if (found == index_.end()) return std::nullopt;
    const Locator loc = found->second;
    const Order removed = *loc.it;
    if (loc.side == Side::Buy) erase_from(bids_, loc);
    else                       erase_from(asks_, loc);
    index_.erase(found);
    return removed;
}
```

`book[o.price]` is the one place where `map::operator[]` is exactly right, because inserting a default `PriceLevel` is what we want. `std::prev(level.fifo.end())` is an iterator to the element just pushed, which is stored so cancel can find it in O(1). `std::optional<Order>` is the return type for "an order, or nothing": `std::nullopt` is the empty value, and the caller tests it with `if (!removed)` and reads it with `*removed`.

Order entry

```
bool submit(Order o, Time now) {
    o.ts = now;
    if (o.type == OrderType::Market) o.tif = TimeInForce::IOC;  // markets
never rest

    if (auto why = validate(o, /*check_duplicate=*/true)) return reject(o,
*why, now);

    if (listener_) listener_->on_accept(o, now);

    if (o.side == Side::Buy) match(o, asks_, now);
    else                     match(o, bids_, now);

    if (o.qty > 0) {  // something is left over
        if (o.type == OrderType::Limit && o.tif == TimeInForce::GTC) {
            if (o.side == Side::Buy) rest(bids_, o);
            else                     rest(asks_, o);
        } else if (listener_) {
            const CancelReason why = o.type == OrderType::Market ?
CancelReason::MarketRemainder
                                                                 :
CancelReason::IocRemainder;
            listener_->on_cancel(o, why, now);
        }
    }
    return true;
}

// Every pre-trade check in one place. Returns the reason to reject, or
nothing.
std::optional<RejectReason> validate(const Order& o, bool check_duplicate)
const {
    if (o.qty <= 0)                                  return
RejectReason::BadQuantity;
    if (o.type == OrderType::Limit && o.price <= 0) return
RejectReason::BadPrice;
    if (o.post_only && (o.type == OrderType::Market || o.tif !=
TimeInForce::GTC))
                                                     return
RejectReason::BadFlags;
    if (check_duplicate && index_.count(o.id) != 0) return
RejectReason::DuplicateId;
    if (o.post_only && would_cross(o))               return
RejectReason::PostOnlyWouldCross;
```

```
    if (o.tif == TimeInForce::FOK && fillable_qty(o) < o.qty)
                                                    return
RejectReason::FokNotFillable;
    return std::nullopt;
}
```

The order of the checks matters. All validation runs *before* the order is accepted or anything in the book changes, so a rejected order never leaves a partial effect. The two checks that peek at the book (post-only and fill-or-kill) also run before matching, because by the time matching starts it is too late to say no. `o` is passed by value, so `o.ts = now` and the later `o.qty` reductions touch only this copy.

After matching, a leftover quantity has exactly two fates: a GTC limit order rests, and everything else (market remainder, IOC remainder) is reported as cancelled. Nothing else can happen, which is easy to check by reading.

Cancel and modify

```
bool modify(OrderId id, Price new_price, Qty new_qty, Time now) {
    auto found = index_.find(id);
    if (found == index_.end() || new_qty <= 0 || new_price <= 0) return
false;

    Order& o = *found->second.it;
    if (new_price == o.price && new_qty <= o.qty) {
        level_of(found->second).total -= (o.qty - new_qty);
        o.qty = new_qty;
        return true;
    }
    const Order old = o;  // copy: remove_resting() destroys the list node o
refers to
    Order replacement = old;
    replacement.price = new_price;
    replacement.qty   = new_qty;
    if (validate(replacement, /*check_duplicate=*/false)) return false;  //
old order stays
    remove_resting(id);
    if (listener_) listener_->on_cancel(old, CancelReason::Replaced, now);
    return submit(replacement, now);  // re-enters at the back of the queue
}
```

Notice the order of operations in the replace path: the new order is validated *before* the old one is removed, so a refused modify leaves the original resting. The first version of this function removed first and validated second, which let a rejected modify destroy the order (the problems section reproduces that bug). `validate()` exists for this reason: one

function holds every pre-trade check, and both `submit` and `modify` call it. The `const Order old = o;` copy is needed because `remove_resting` destroys the list node that `o` refers to.

This encodes a rule that is nearly universal on real venues: reducing size at the same price keeps your place in the queue, and anything that could disadvantage the people behind you (a new price, or more size) sends you to the back. Without that rule, a trader could rest a tiny order to get front-of-queue and then inflate it.

Proving it works

`check_invariants()` re-derives the structural facts from scratch: the book is not crossed, no level is empty, each cached total equals the sum of its orders, and every resting order is findable by id and the id points to that very order. `tests.cpp` calls it every thousand operations of a 200,000-operation random run, with self-trade prevention both on and off, and the tests also check price priority, time priority, partial fills, IOC, FOK and post-only individually. All 483 checks pass, including when compiled with the address and undefined-behaviour sanitizers (`-fsanitize=address,undefined`).

Part 3: The engine and order types (`engine.hpp`)

The book knows how to match. The **engine** is the exchange's front door: it decides what is allowed to reach the book, hands out order ids, keeps market statistics and checks who owns what. Keeping these jobs out of `OrderBook` means the book stays small enough to verify completely.

Order types in the book

The five order types from the anatomy section reduce to three flags on `Order`, and the book's behaviour follows from them:

Order	`type`	`tif`	`post_only`	After matching, leftover quantity…
Limit	Limit	GTC	false	rests at its limit price
Market	Market	IOC (forced)	false	is cancelled (`MarketRemainder`)
IOC	Limit	IOC	false	is cancelled (`IocRemainder`)
FOK	Limit	FOK	false	never exists: the order is rejected up front unless the whole quantity is available

Order	`type`	`tif`	`post_only`	After matching, leftover quantity…
Post-only	Limit	GTC	true	rests, because the order is rejected up front if it would trade

Fill-or-kill needs a look-ahead, `fillable_qty()`, which walks the opposite side level by level and adds up quantity until the target is reached or the price stops crossing. With self-trade prevention on it skips the same owner's orders, since those would be cancelled rather than traded. Without that detail, a FOK could be accepted, then partly cancelled by STP, and end up partly filled, which breaks the "all or nothing" promise. The test test_fok_with_stp checks exactly this case: a FOK that could only fill by trading against its owner's own order is rejected and leaves the book untouched.

The engine class

```
struct RiskLimits {
    Qty    max_order_qty = 1'000;
    double price_band    = 0.10;  // reject limit prices more than 10% from
the reference
};

class Engine : public Listener {
public:
    explicit Engine(Listener* downstream = nullptr, RiskLimits limits = {},
                    bool self_trade_prevention = true)
        : book_(this, self_trade_prevention), downstream_(downstream),
limits_(limits) {}

    OrderId allocate_id() { return ++last_id_; }

    bool submit(Order o, Time now) {
        if (!passes_risk(o, now)) return false;
        return book_.submit(o, now);
    }

    // Only the owner may cancel or modify an order.
    bool cancel(AgentId requester, OrderId id, Time now) {
        const Order* o = book_.find(id);
        if (o == nullptr || o->owner != requester) return false;
        return book_.cancel(id, now);
    }
    ...
    void on_trade(const Trade& tr) override {
        last_price_      = tr.price;
        reference_price_ = tr.price;
```

```
        volume_         += tr.qty;
        ++trades_;
        if (downstream_) downstream_->on_trade(tr);
    }
```

Things to notice:

- `class Engine : public Listener` means the engine *is a* listener. It passes itself to the book (`book_(this, ...)`), so the book reports to the engine first, the engine updates its statistics, and then it forwards the event downstream. This is a chain of observers, and it is how the simulator learns about trades without the book knowing the simulator exists.
- `override` makes the compiler verify that `on_trade` really replaces the base class's method. Misspell it and the build fails instead of silently doing nothing.
- `const Order* o = book_.find(id)` returns a pointer to the resting order or `nullptr`. The pointer is valid only until the next change to the book, so it is used immediately and never stored.
- Ownership checks live in the engine, not the book. The book trusts whoever calls it, as a matching core should; the gateway in front of it authenticates.
- The price band compares each limit price with a **reference price**, which starts as a configured value and then follows the last trade. Orders more than 10% away are rejected. This is a crude version of the limit-up/limit-down and fat-finger checks real venues apply.

A bug found by reading the code

The first version of `Engine::modify` forwarded straight to the book, so a trader could dodge the size limit and price band by placing a legal order and then modifying it to an illegal one. The fix is that `submit` and `modify` now share one `passes_risk()` function, and `tests.cpp` checks that an oversized or out-of-band modify is refused and leaves the order untouched. The general lesson: every path that can create or change a live order must pass through the same validation, and the safest way to guarantee that is to have only one such path.

What the engine does not do

It has no authentication, no per-participant rate limit, no kill switch, no clearing or settlement, and no persistence. The problems section returns to what that costs.

Port it: a gateway object with `submit / cancel / modify`, one shared validation function, a monotonically increasing id counter and an observer hook. Everything else is a policy detail.

Part 4: Market participants (`agents.hpp`)

An order book with nobody trading in it is a data structure. A market needs participants who want different things. This file builds four, each a small class with one decision rule, and each representing a real category of market behaviour.

The one rule: agents act through a Context

```
class Context {
public:
    virtual ~Context() = default;
    virtual Time                    now() const = 0;
    virtual Engine::Quote           quote() const = 0;   // top of book + last
trade
    virtual std::vector<LevelInfo> depth(Side, std::size_t) const = 0;
    virtual OrderId limit(Side, Price, Qty, TimeInForce, bool post_only) = 0;
    virtual OrderId market(Side, Qty) = 0;
    virtual void    cancel(OrderId) = 0;
    virtual std::mt19937_64& rng() = 0;
};
```

`= 0` after a virtual method makes it *pure*: the class is abstract, it cannot be instantiated, and any subclass must supply the method. That is C++'s way of writing an interface. An agent never holds a pointer to the book. It can read the public quote and send requests, and the simulator decides when those requests arrive, which is how latency gets modelled without any agent knowing about it. Be clear about the simplification: `quote()` shows the exchange's current state with **no delay**, and only orders, cancels and fill notifications travel with latency. A more faithful model would also delay the market data feed, and the problems section lists that. The same agent code could drive a live exchange connection if you wrote a `Context` that talked to one.

All randomness comes from `rng()`, one seeded generator shared by the whole simulation. That is what makes a run reproducible.

The agent base class

```
class Agent {
public:
    Agent(AgentId id, std::string name) : id_(id), name_(std::move(name)) {}
    virtual ~Agent() = default;

    // Returns how long until the next wake-up in nanoseconds (negative =
never again).
    virtual Time on_wake(Context& ctx) = 0;
```

```
    virtual void on_fill(Context&, const Trade&, bool /*as_maker*/) {}

    void record_fill(const Trade& t, bool as_maker) {
        const Side mine = as_maker ? opposite(t.taker_side) : t.taker_side;
        const Qty  signed_qty = mine == Side::Buy ? t.qty : -t.qty;
        position += signed_qty;
        cash     -= signed_qty * t.price;  // buying spends cash
        ++fills;
        traded += t.qty;
    }
    ...
    Qty          position = 0;  // signed inventory
    std::int64_t cash     = 0;  // in ticks * units
    double       edge     = 0;
};
```

Every agent is driven by timers: the simulator calls `on_wake`, the agent acts, and returns the delay until its next wake-up. `std::move(name)` hands the string's buffer over instead of copying it. The bookkeeping is the same for every agent: inventory (`position`) and `cash`. **Mark-to-market profit** is `cash + position * true_value`, so an agent that bought 10 units for 1000 each and now sees 1010 is up 100 even though it has not sold.

`edge` is a second, sharper measure. For each fill it adds `qty * (true_value - price)` for a buy and the reverse for a sell, using the hidden true value at the moment of the trade. It isolates how good each trade was from how the inventory happened to move afterwards, and it is what the results section uses to tell skill from luck.

1. Noise trader: uninformed flow

In finance, noise traders trade for reasons unrelated to the asset's value: rebalancing, liquidity needs, whim. They are the reason a market maker can make money, because their flow carries no information.

```
Time on_wake(Context& c) override {
    auto& rng = c.rng();
    const Side side = std::bernoulli_distribution(0.5)(rng) ? Side::Buy :
Side::Sell;
    const Qty  qty  = std::uniform_int_distribution<Qty>(1, max_qty_)(rng);
    const double u  = std::uniform_real_distribution<double>(0.0, 1.0)(rng);

    if (u < 0.25) {                                  // 25%: market order
        c.market(side, qty);
    } else if (u < 0.90) {                           // 65%: limit order
near the price
        const Price off = 1 + std::geometric_distribution<Price>(0.3)(rng);
```

```
        const Price ref = reference_price(c.quote(), initial_);
        const Price px  = side == Side::Buy ? ref - off : ref + off;
        live_.push_back(c.limit(side, px, qty, TimeInForce::GTC, false));
        if (live_.size() > kMaxLive) {                // keep the footprint
bounded
            c.cancel(live_.front());
            live_.pop_front();
        }
    } else if (!live_.empty()) {                      // 10%: cancel the
newest order
        c.cancel(live_.back());
        live_.pop_back();
    }
    return exp_wait(rng, mean_wake_ms_);
}
```

C++ point: a random distribution is an object, `std::bernoulli_distribution(0.5)`, which you call *with* a generator, `(rng)`, to draw a value. The engine (`mt19937_64`) produces raw bits and the distribution shapes them. `exp_wait` draws an exponentially distributed delay, which makes arrivals a **Poisson process**: events at a steady average rate but with memoryless random gaps, the standard model for independent traders. Passing the generator by `auto&` reference matters: a copy would replay the same numbers every time.

2. Market maker: quotes both sides and manages inventory

A market maker earns the spread by posting a bid and an ask and hoping both fill. The risk is inventory: after buying more than it sells it is long, and a falling price hurts. So it shifts its quotes against its position, which is called **skewing**.

```
Time on_wake(Context& c) override {
    if (bid_id_) c.cancel(bid_id_);   // pull the old quotes ...
    if (ask_id_) c.cancel(ask_id_);
    bid_id_ = ask_id_ = 0;

    const Price fair = reference_price(c.quote(), initial_);
    // Long inventory: shift both quotes down so we sell more easily (and
vice versa).
    const Price shift = static_cast<Price>(std::lround(skew_ *
static_cast<double>(position)));
    const Price bid = fair - half_spread_ - shift;
    const Price ask = fair + half_spread_ - shift;

    // ... and post new ones. Post-only: we only ever want to be the maker.
    if (position <  max_inv_) bid_id_ = c.limit(Side::Buy,  bid, size_,
TimeInForce::GTC, true);
    if (position > -max_inv_) ask_id_ = c.limit(Side::Sell, ask, size_,
TimeInForce::GTC, true);
    return refresh_;
}
```

It cancels and re-quotes on a fixed timer, always post-only so it can never accidentally take liquidity, and it stops quoting one side entirely when its inventory limit is reached. Its notion of fair value is simply the current mid-price, which is a weakness discussed later: it learns nothing from outside the book. `static_cast<Price>(...)` is C++'s explicit conversion, required here because floating-point to integer conversion loses information.

3. Momentum trader: follows the trend

It keeps two exponential moving averages of the mid-price, a fast one and a slow one, and trades in the direction of their gap.

```
fast_ += 0.30 * (mid - fast_);   // exponential moving averages
slow_ += 0.05 * (mid - slow_);
const double signal = fast_ - slow_;
// IOC limit a couple of ticks through the touch: a "protected" market order.
if (signal > threshold_ && position < max_pos_)
    c.limit(Side::Buy,  *q.ask + 2, size_, TimeInForce::IOC, false);
else if (signal < -threshold_ && position > -max_pos_)
    c.limit(Side::Sell, *q.bid - 2, size_, TimeInForce::IOC, false);
```

The IOC limit two ticks through the touch is a *protected* market order: it behaves like a market order but cannot sweep the book at any price. Traders and brokers use this form when they want certainty of execution with a worst-case price.

4. Informed trader: knows something the book does not

This agent receives a noisy reading of the hidden true value through a callback and trades only when the book's price is far enough from it to cover the cost of crossing.

```
Time on_wake(Context& c) override {
    const double v = signal_();  // the private information
    const Engine::Quote q = c.quote();
    if (q.ask && position < max_pos_ && v - static_cast<double>(*q.ask) >=
edge_)
        c.limit(Side::Buy,  *q.ask, std::min(size_, q.ask_qty),
TimeInForce::IOC, false);
    else if (q.bid && position > -max_pos_ && static_cast<double>(*q.bid) - v
>= edge_)
        c.limit(Side::Sell, *q.bid, std::min(size_, q.bid_qty),
TimeInForce::IOC, false);
    return exp_wait(c.rng(), mean_wake_ms_);
}
```

The `std::function<double()>` member holds any callable returning a double: here a lambda that reads the simulator's hidden value and adds Gaussian noise. The informed trader is the reason the price moves toward the truth: its buying lifts the quotes when the price is too low, and its selling pushes them down when it is too high. In economics this is the adverse selection that market makers fear, because the other side of an informed trade is on average a loser.

Port it: an agent is an object with a wake-up method that reads a market snapshot and issues requests, plus a few lines of bookkeeping. Write each agent's rule as a function from a market snapshot to a list of orders, and you can reuse the same agents on any simulator, backtester or live gateway.

Part 5: The simulator (`sim.hpp`, `main.cpp`)

The simulator is a **discrete-event simulation**. Instead of ticking a clock every microsecond, it keeps a priority queue of future events ordered by time, repeatedly pops the earliest one, sets the clock to its timestamp and runs it. Nothing happens between events, so simulating a minute of trading costs as much as the events in that minute, not 60 billion clock ticks.

The event queue

```
struct Event {
    Time                  t;
    std::uint64_t         seq;  // tie-breaker: same time, first scheduled
runs first
    std::function<void()> fn;
};
struct Later {  // "a has lower priority than b" turns std::priority_queue
into a min-heap
    bool operator()(const Event& a, const Event& b) const {
        return a.t != b.t ? a.t > b.t : a.seq > b.seq;
    }
};
void at(Time t, std::function<void()> fn) { queue_.push(Event{t, seq_++,
std::move(fn)}); }

void run() {
    at(0, [this] { fundamental_step(); });
    at(0, [this] { log_quote(); });
    while (!queue_.empty()) {
        Event e = queue_.top();  // copy: top() is const
        queue_.pop();
        now_ = e.t;
        e.fn();
    }
}
```

The loop at the bottom is the entire engine of the simulation. Three C++ details are worth learning here:

- `std::priority_queue` is a *max*-heap by default, so the comparator `Later` says an event with a later time has *lower* priority, which makes the earliest event come out first. This inversion trips up nearly everyone once.
- `seq` breaks ties. Without it two events with the same timestamp would run in an arbitrary order, and the run would not be reproducible. With it, same-time events run in the order they were scheduled.
- An event is a closure. `[this] { fundamental_step(); }` is a lambda that captures the simulator pointer, and `std::function<void()>` stores it. Closures that outlive the code that created them must capture by value, which is why you will see `[this, id]` and `[this, o]` and never `[&]` when scheduling.

Latency: every action becomes a future event

```
OrderId send(Order o) {
    o.id    = sim_.engine_.allocate_id();  // the id exists now; the order
arrives later
    o.owner = id_;
    sim_.at(sim_.now_ + latency(), [this, o] { sim_.engine_.submit(o,
sim_.now_); });
    return o.id;
}
void cancel(OrderId id) override {
    sim_.at(sim_.now_ + latency(), [this, id] { sim_.engine_.cancel(id_, id,
sim_.now_); });
}
```

When an agent calls `limit(...)`, nothing touches the book yet. The simulator allocates the order's id immediately, so the agent can refer to it, and schedules the real submission one latency later. The same applies to cancels, and to fill notifications going back the other way (`deliver` in `sim.hpp`). Because a cancel and the order it targets travel with the same latency and the cancel is scheduled later, the order always arrives first, so an agent cannot cancel something that does not exist yet.

This tiny mechanism is what produces every latency effect in real markets. An agent with 50 microseconds of latency sees a quote, decides, and its order lands after the market has had 50 microseconds to move. A slower agent loses races for the same liquidity.

Creating agents: a variadic template

```
template <class T, class... Args>
T* add(Time latency, Time first_wake, Args&&... args) {
    const AgentId id = static_cast<AgentId>(slots_.size() + 1);
    auto agent = std::make_unique<T>(id, std::forward<Args>(args)...);
    T* raw = agent.get();
    slots_.push_back(Slot{std::move(agent), latency,
std::make_unique<AgentContext>(*this, id)});
    schedule_wake(id, first_wake);
    return raw;
}
```

This is the most advanced C++ in the project, and it solves a common problem: create an object of *any* agent type, passing along *whatever* constructor arguments it takes. `class... Args` is a parameter pack, a list of any number of types, and `std::forward`

passes each argument on without extra copies. The caller writes `sim.add<MarketMaker>(50 * US, 0, refresh, 2, 10, 100, 0.05, p0)` and the compiler checks the arguments against `MarketMaker`'s constructor at build time.

`std::unique_ptr` owns each agent. The simulator is the only owner, so agents are destroyed with it, and no one has to remember to delete anything. `add` returns a raw pointer for the caller to *observe* the agent afterwards, which is safe only while the simulator is alive.

Port it: in a language without variadic templates, take a factory function `(id) -> Agent` instead. Closures capturing the constructor arguments give you the same flexibility.

The hidden true value

```
void fundamental_step() {
    fund_ = std::max(1.0, fund_ + std::normal_distribution<double>(0.0,
cfg_.fundamental_sigma)(rng_));
    if (now_ + cfg_.fundamental_step <= cfg_.duration)
        at(now_ + cfg_.fundamental_step, [this] { fundamental_step(); });
}
```

Every 10 ms the true value takes a Gaussian step of standard deviation 0.25 ticks, a random walk. No agent reads it directly. The informed traders see it through noise (standard deviation 1 tick), and the simulator uses it to measure how well the market tracks it (`mean |mid - true value|`) and to compute each agent's `edge`. A simulator that knows the truth can grade the market, which a real market never can.

Determinism

There is one random generator, seeded from `SimConfig::seed`, and one event order fixed by `(time, seq)`. Running `./sim 42` twice gives byte-identical output, and `tests.cpp` checks that the same seed gives the same trade count and volume while a different seed does not. Reproducibility is what lets you debug a surprising market by replaying it, and compare two configurations on the same random draws.

Two habits preserve it: never use the system clock or `rand()`, and never iterate over an `unordered_map` in a way that affects behaviour, because its order is unspecified. The simulator's report uses an ordered `std::map` for exactly that reason.

Assembling a market

```
sim.add<MarketMaker>(mm_latency, 0, /*refresh*/ mm_refresh, /*half_spread*/
2, /*size*/ 10,
                     /*max_inventory*/ 100, /*skew*/ 0.05, p0);

for (int i = 0; i < 20; ++i)
    sim.add<NoiseTrader>(1 * MS, (i + 1) * MS, /*mean_wake_ms*/ 50.0,
/*max_qty*/ 10, p0);

for (int i = 0; i < 3; ++i)
    sim.add<MomentumTrader>(500 * US, (i + 1) * 7 * MS, /*mean_wake_ms*/
20.0, /*size*/ 5,
                            /*max_pos*/ 50, /*threshold*/ 1.5);

for (int i = 0; i < 2; ++i)
    sim.add<InformedTrader>(200 * US, (i + 1) * 11 * MS, /*mean_wake_ms*/
30.0,
                            /*size*/ 5, /*max_pos*/ 2000, /*edge*/ 3.0,
                            [&sim] { return sim.noisy_fundamental(); });
```

One market maker, twenty noise traders, three momentum traders and two informed traders, with the market maker the fastest (50 microseconds), the informed traders next (200 microseconds), then momentum (500 microseconds) and noise (1 millisecond). Run it with `./sim 42` and it prints a report; it also writes `trades.csv` and `quotes.csv` (time, best bid, best ask, mid, true value, last price) for plotting.

Testing and results

The book passes 483 checks in `tests.cpp`, and the simulated market behaves the way market microstructure theory says it should: informed traders profit, noise traders lose a little on every trade, and the market maker earns a steady small amount. All numbers below were measured on this code. Build and run everything with:

```
g++ -std=c++17 -O2 -Wall -Wextra -o tests tests.cpp && ./tests
g++ -std=c++17 -O2 -Wall -Wextra -o sim main.cpp && ./sim 42
./sim sweep 100          # 100 seeds, averaged
g++ -std=c++17 -O2 -Wall -Wextra -o bench bench.cpp && ./bench
```

What the tests cover

Test	What it proves
Rest, best price, price priority, time priority	The two matching rules, including that makers keep the better price
Partial fills, market, IOC, FOK, post-only	Each order type's leftover handling and rejection rules
Cancel and modify	Shrinking keeps queue position; growing or repricing loses it; a refused modify leaves the original resting
Self-trade prevention, and FOK combined with it	Own orders are cancelled, never traded, and FOK counts only tradable size
Rejects and engine risk checks	Bad quantity, bad price, duplicate id, size limit, price band, ownership, and that modify cannot bypass limits
Random invariants	200,000 random operations, with STP on and off, checking `check_invariants()` throughout
Differential	The map-based book and the flat-array book agree on every accept, cancel, best price and trade over 300,000 random operations (186,281 trades)
Simulator determinism	Same seed, same result; different seed, different result

The differential test is the most valuable idea here. When you have two implementations of one specification, one slow and obviously correct and one fast and tricky, run them on the same random input and compare everything. Most optimisation bugs surface within seconds. The suite also runs clean under `-fsanitize=address,undefined`, which catches use-after-free and signed overflow that normal runs hide.

What the simulation shows

Averaged over 100 seeds of 60 simulated seconds each, with the market maker at 50 microseconds latency refreshing every 5 ms:

Agent type	Count	Mean PnL per agent per run ($)	Edge per unit traded (ticks)
Informed	2	+89.75	+5.15
Market maker	1	+27.53	+0.88
Momentum	3	-2.52	-0.93
Noise	20	-9.97	-0.38

The market's mean spread was 2.49 ticks and its mid-price stayed on average 6.35 ticks from the hidden true value. Several things are worth checking against the table:

- **PnL sums to zero.** 2 × 89.75 + 27.53 + 3 × (-2.52) + 20 × (-9.97) = 0.07, which is rounding. Trading is zero-sum, so a simulator whose agent PnLs do not sum to zero has a bookkeeping bug. This check caught nothing here, but it is the first thing to run on any matching engine.
- **Informed traders profit at the expense of the others.** Their edge is positive on average because they trade only when the book is at least 3 ticks from their noisy estimate of the true value. The noise and momentum traders lose most, and the market maker gives some of its spread income back on the trades where it was the one picked off.
- **The market maker's edge is positive but small** (under a tick per unit) and it comes from the noise traders. In real markets a maker's edge is roughly the spread it captures minus what it loses to informed flow, and this model shows the same structure.
- **Single runs are noisy.** On seed 42 the informed traders' PnL was **-$204.81 each** even though their edge per unit was +4.21. They finished that run holding large inventory marked to a final price that moved against them. Edge is a property of the trades, and PnL adds inventory risk on top, so never judge a strategy from one seed.

Does the market maker's speed matter?

I expected a faster market maker to lose less to informed traders and tested it by changing the market maker's latency and quote-refresh interval (100 seeds each, other agents unchanged):

MM latency	MM refresh	Informed PnL/agent ($)	MM PnL ($)	Mean spread (ticks)	Mean distance to true value (ticks)
50 µs	1 ms	153.31	20.19	2.49	7.71
50 µs	5 ms	89.75	27.53	2.49	6.35
1 ms	5 ms	88.99	28.08	2.48	6.56
5 ms	20 ms	72.93	28.88	2.50	6.29

MM latency	MM refresh	Informed PnL/agent ($)	MM PnL ($)	Mean spread (ticks)	Mean distance to true value (ticks)
20 ms	50 ms	89.28	26.93	2.54	6.21

The result is that, **in this model, speed barely mattered**: the maker's profit stayed between about $27 and $29 across a 400-fold change in latency, and the one row that differs (refreshing every millisecond) is *worse* for the maker, not better. I did not compute confidence intervals, so treat differences of a few dollars as noise, and I did not investigate why the 1 ms refresh row differs. The likely reason that speed does not help is a limit of the model, not a fact about markets: the maker's fair value is the book's own mid-price, so it has no information with which to dodge an informed order, and the informed traders see the book instantly in this model. Real latency races arise from information that arrives at different times, which this simulator does not model. The problems section returns to that.

What the tests do not cover

There is no live or multithreaded test, because the book is single-threaded by design. The benchmark and simulator were checked for crashes and for agreement between the two books, but not against an independent reference exchange. A production engine would also be replayed against recorded exchange data and fuzzed with malformed messages.

Problems with the final product

The product is a correct teaching matching engine and a deliberately simple market model. It is not an exchange, and it is not a realistic market. The problems below are ordered by how much they would hurt in real use. Rows marked **ran** were reproduced by running code for this tutorial; the rest come from reading the code.

Correctness and robustness

#	Problem	What happens	Evidence
1	Modify was not atomic (now fixed)	`OrderBook::modify` removed the old order before validating the new one. If the new one is rejected (for example a post-only order repriced so that it would cross) the original is already gone and the caller gets `false`	**ran**: a resting post-only bid moved to a crossing price returned `false` and the order had vanished. Fixed in the delivered code by validating first

#	Problem	What happens	Evidence
2	The price band can be walked	The band is centred on the last trade price. Two cooperating accounts can trade at +9% per step, and every order is inside the band	**ran**: 10 steps moved the price 2.4× (10,000 to 23,670 ticks) while a single order at 2× was refused
3	Market orders ignore the band	`passes_risk` checks only limit orders, so a market order will sweep every level it can reach at any price	Read from the code
4	No rate limits	A participant can submit and cancel orders without limit (quote stuffing), with no order-to-trade ratio or message throttle	Read from the code
5	No sequence numbers or persistence	Events carry no global sequence number, there is no journal, and a crash loses every resting order. Consumers cannot detect a gap	Read from the code
6	Rejects are not delivered to agents	Agents get fills but not accepts, rejects or cancel confirmations, so a market maker whose post-only quote was rejected still believes it is quoting	Seen in the first run: 12 `PostOnlyWouldCross` rejects, none visible to the agent
7	One symbol, one thread	Nothing handles several instruments, and the book has no locks	By design
8	No auctions, halts or circuit breakers	Opens and closes simply start continuous trading, and a crash in price cannot pause the market	By design
9	One self-trade policy	Only "cancel the resting order" is implemented	By design

Problem 2 is the most interesting because every individual check passes. Each order is valid and the combination is an attack. Validation that looks at one order at a time cannot see a pattern, which is why real venues add surveillance on top.

Performance limits of the map-and-list design

Problem	Why	Measured
One heap allocation per resting order	Every `std::list` node is a separate `malloc`, scattered across memory	160 ns to rest an order and 293 ns to cancel one, against 40 and 36 ns on the flat book (one run; ratios were stable across three)
Pointer chasing	Walking a queue or the tree follows pointers to arbitrary addresses, so the CPU's cache and prefetcher help little	Sweeping about 10 orders cost 2,260 to 3,218 ns across three runs, roughly 230 to 325 ns per fill
A virtual call per event	`Listener` calls cannot be inlined	Not isolated in the benchmark; the flat book also removes it, so the speed-up includes it
Rare large stalls	A growing hash map or tree rehash and page faults show up as outliers	Maximum latency was roughly 0.7 to 1.5 ms on both books. I did not identify the cause, and on a shared cloud machine it may well be the operating system

Problems in the flat-array book (`fast_book.hpp`)

The fast book is a benchmark vehicle and an illustration, and it skips protections the first book has. Three were reproduced:

- **A huge order id crashes it** (**ran**). The id map is a vector indexed by id, so resting an order with id 2^{40} asks for a terabyte-scale allocation and throws `std::bad_alloc`. Anyone who can choose ids can take the process down.
- **A duplicate id corrupts it** (**ran**). It does not check for a duplicate, so the second order with the same id overwrites the map entry. After one `cancel(5)`, the first order remained at the best bid and could never be cancelled.
- **Fixed price range.** Out-of-range limit prices are silently refused. It also lacks self-trade prevention, fill-or-kill, post-only and modify, and its differential test runs with the main book's self-trade prevention switched off for that reason.

Problems with the simulated market

Limitation	Consequence
The market maker's fair value is the book's own mid-price	It cannot see the true value, so it can only react after informed traders have already moved the book. Price discovery rests entirely on the informed traders
Price discovery is fragile	Capping each informed trader at 200 units of inventory instead of 2,000 left the mid-price 26 ticks from the true value on average, against 6.4 ticks with the larger cap (20 seeds each, same market maker). One parameter changed the market's quality four-fold
Market data has no latency	Agents read the book instantly, so the model cannot reproduce latency arbitrage. The speed experiment above found no benefit for the fastest maker, which is the symptom
No fees or rebates	Post-only orders have no economic reason to exist, and the maker's profit ignores the maker-taker fee structure real venues use
Poisson arrivals and uniform sizes	Real order flow is bursty and heavy-tailed, with clusters of activity and a few very large orders
A Gaussian random-walk true value	No jumps, no news, no volatility clustering
Agents never learn	Every parameter is fixed, so no strategy adapts to another
One venue	No fragmentation, no routing, no cross-venue arbitrage

The results from this simulator are therefore *qualitatively* sensible (informed traders win, noise traders pay a small cost, the maker earns a thin spread) but should not be used to estimate real profits, real impact or real risk.

Improvements and how to implement them

Each improvement below names the problem it removes. Three of them (rate limiting, a market-order collar, and journal-and-replay) come with code that was compiled and run for this tutorial; they are in `improve_test.cpp` with checks. The rest are outlines with the algorithm to follow, not tested code.

Improvement	Fixes	Effort
Validate before removing in modify	Problem 1 (done, see Part 2)	Small
Market-order collar	Problem 3	Small
Rate limits per participant	Problem 4	Small
Journal inputs, sequence every event	Problem 5	Medium
Slow-moving reference price and trading halts	Problem 2	Medium
Deliver acks, rejects and cancels to agents	Problem 6	Small
Stop, iceberg and pegged orders; auctions	Problem 8, plus missing order types	Large
One book per symbol, one thread per book	Problem 7	Medium
Safer ids in the flat book	Flat-book problems	Small
A more realistic simulator	Simulator problems	Medium to large

Market-order collar (Problem 3)

Turn every market order into an IOC limit order a fixed distance from the reference price. It still behaves like a market order for ordinary sizes, and it cannot walk the book to absurd prices. Many venues call this market-to-limit with a collar or protection points.

```
inline Order with_collar(Order o, Price reference, double band) {
    if (o.type != OrderType::Market) return o;
    const Price room = static_cast<Price>(static_cast<double>(reference) *
band);
    o.type  = OrderType::Limit;
    o.tif   = TimeInForce::IOC;
    o.price = o.side == Side::Buy ? reference + room : reference - room;
    return o;
}
```

Call it inside `Engine::submit` before the risk check. The check confirmed that a 15-unit market buy with a 5% collar around 10,000 filled only the 5 units offered at 10,500 and left the level at 10,900 untouched.

Rate limits (Problem 4)

A token bucket allows bursts but caps the sustained rate. Each participant gets a bucket that refills at a fixed rate; every order, cancel and modify spends one token, and an empty bucket means rejection.

```
class TokenBucket {
public:
    TokenBucket(double per_second, double burst) : rate_(per_second),
cap_(burst), tokens_(burst) {}
    bool allow(Time now) {
        tokens_ = std::min(cap_, tokens_ + rate_ * static_cast<double>(now -
last_) / static_cast<double>(SEC));
        last_ = now;
        if (tokens_ < 1.0) return false;
        tokens_ -= 1.0;
        return true;
    }
private:
    double rate_, cap_, tokens_;
    Time   last_ = 0;
};
```

The check showed a bucket of 5 tokens refilling at 10 per second allowing exactly 5 orders at time zero, one more after 100 ms, and never more than 5 after a long pause. In the engine, keep `std::unordered_map<AgentId, TokenBucket>`, call `allow(now)` first in `submit`, `cancel` and `modify`, and add a `RateLimit` reject reason. Cancels need their own, more generous limit: throttling them too hard stops participants from pulling stale quotes, which is itself dangerous. Real venues also cap the ratio of orders sent to trades done.

Journal the inputs, replay to recover (Problem 5)

Because the engine is deterministic, you do not need to store its state. Store the *inputs* in arrival order and rebuild the state by feeding them to a fresh engine. This is event sourcing, and it is how many exchange cores recover from a crash and how they are tested against production traffic.

```
struct Input {
    enum Kind : std::uint8_t { Submit, Cancel, Modify } kind;
    Time    t;
    Order   order;   // for Submit
    OrderId id;      // for Cancel / Modify
    Price   price;   // for Modify
    Qty     qty;     // for Modify
};

static void apply(OrderBook& b, const Input& in) {
    switch (in.kind) {
        case Input::Submit: b.submit(in.order, in.t); break;
        case Input::Cancel: b.cancel(in.id, in.t); break;
        case Input::Modify: b.modify(in.id, in.price, in.qty, in.t); break;
    }
}
```

In the check, 200,000 random inputs were applied to one book while being logged, then replayed into a second book. Both produced the same 75,344 trades, in the same order, and ended with the same 6,341 resting orders and the same best prices. For production you would also (1) give every input and every output a global sequence number so consumers can detect gaps, (2) write the journal to disk before acting on it, and (3) take periodic snapshots so recovery replays minutes, not days. The replay requirement adds a rule for the engine itself: nothing inside it may read the wall clock, a random number or an unordered iteration order.

Reference price, halts and auctions (Problems 2, 8)

The band-walking attack works because the reference follows the last trade. Three standard defences, in increasing strength:

1. **Slow the reference.** Use a time-weighted or volume-weighted average over a window, or the previous close, so a chain of small trades cannot drag it.
2. **Halt on volatility.** If the price moves more than X% within Y seconds, stop continuous trading for a fixed pause. Implement it as a state on the engine (`Open`, `Halted`) that rejects or queues orders.

3. **Call auctions.** During a halt, and at the open and close, collect orders without matching. Then pick the single price that maximises executable volume: for each candidate price p, the matchable quantity is `min(buy quantity at or above p, sell quantity at or below p)`; choose the p with the largest value, breaking ties by smallest imbalance and then by closeness to the reference. Match everything at p in priority order. This needs only the existing book plus a pass over its levels.

Richer order types

- **Stop and stop-limit.** Keep a second structure for untriggered orders, buy stops sorted ascending by trigger and sell stops descending. After each trade changes the last price, release every stop whose trigger has been crossed as a market or limit order. Process releases from a queue, not recursively, because a released order can trade and trigger more stops.
- **Iceberg.** Store a displayed quantity and a hidden reserve. When the displayed part fills, refill it from the reserve and send the order to the back of the level's queue. Most venues give the refreshed slice a new timestamp.
- **Pegged.** An order whose price is a function of the best bid or ask. It must be re-priced when the touch moves, so the book needs a list of pegged orders to update.

Deliver every event to agents (Problem 6)

The simulator already receives `on_accept`, `on_reject` and `on_cancel` from the engine through the `Listener` interface. Forward them to the owning agent with the same latency model as fills, and give `Agent` matching virtual methods. A market maker that learns its post-only quote was rejected can re-quote immediately instead of waiting for its timer.

Threads and symbols (Problem 7)

Do not make the book thread-safe with locks. Give each symbol its own book and let exactly one thread own each book (the *single-writer principle*). Inputs arrive through a lock-free single-producer ring buffer, outputs leave through another, and the matching path takes no locks at all. Scale by sharding symbols across threads. This is the architecture of most low-latency exchange cores, because a lock costs more than the whole match.

Safer ids in the flat book

Both reproduced flaws disappear if ids cannot be chosen by clients. `Engine::allocate_id()` already hands out dense increasing ids, and a flat book behind it cannot see a duplicate or a huge id. If clients must supply ids, put an open-addressing

hash map from id to pool index inside the flat book and reject duplicates at the door.

A more realistic simulator

Measure before changing. On ten seeds, the simulated mid-price sampled every 100 ms shows two features of real markets already: returns are fat-tailed (excess kurtosis about 150, driven by 19 jumps of more than 5 ticks among 5,990 returns, against a standard deviation of about 1 tick), and volatility clusters (the lag-1 autocorrelation of absolute returns is 0.33). Kurtosis is dominated by a few events and I did not establish why the jumps occur, so treat it as a rough indicator. Before trusting the simulator for anything quantitative, extend it in this order and re-measure each time:

1. **Delay the market data.** Keep a short history of quotes and have `Context::quote()` return the one from `now - feed_latency`. This is what makes latency arbitrage possible at all.
2. **Give the market maker information.** A second, lagged price signal (another venue, a correlated asset) plus the Avellaneda-Stoikov reservation price `r = s - qγσ²(T - t)`, where s is the mid, q the inventory, γ the risk aversion and σ the volatility. It widens quotes when volatility rises and skews them against inventory.
3. **Model fees.** Charge takers and pay makers a rebate in `record_fill`. Post-only orders then have a reason to exist.
4. **Burstier flow.** Replace Poisson arrivals with a self-exciting Hawkes process and draw sizes from a log-normal or Pareto distribution.
5. **A jump process for the true value** and scheduled news events.
6. **Check stylized facts as tests.** Fat tails, clustering, long memory in order signs, and a concave (roughly square-root) price impact of large orders. A failing simulator should break the build, like any other test.

Optimisation

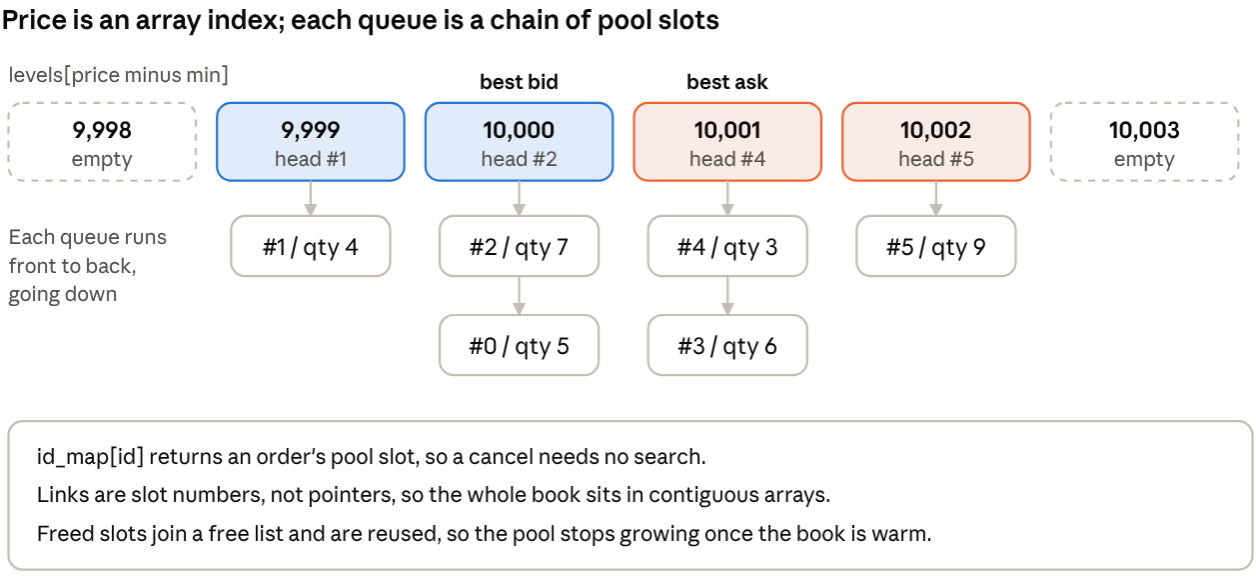
On the same operation streams, a flat-array book with a pooled order store ran **2.9 to 8.3 times faster** than the map-and-list book, with the largest gain on cancels. It is a measured result from this machine, and the section also says what the flat book gives up to get it.

Where the time goes

I did not run a profiler, so the causes below are the usual suspects and not measurements. What was measured is the combined effect. The map-and-list book pays for four things on every operation: a heap allocation for each list node, pointer-chasing through tree and list

nodes that sit at unrelated addresses, a hash-map lookup per order, and a virtual call per event. Each can be removed by a different design choice, and the flat book removes all four at once.

The flat book (`fast_book.hpp`)



FastBook layout · 6 price levels, 6 resting orders, illustrative

- **Price is an index.** The levels are two arrays, `bids_` and `asks_`, indexed by `price - min`. Finding a level is arithmetic, not a tree walk. The best bid and ask are kept as two integers.
- **Orders live in one pool.** Every resting order is a slot in a single `std::vector<Node>`. A queue is a chain of slot numbers (`prev` , `next`), not pointers, and freed slots form a free list.
- **The id map is an array.** `id_map_[id]` holds the order's slot, or -1.
- **The sink is a template parameter.** `FastBook<Sink>` calls `sink_.on_trade(...)` on a type known at compile time, so the compiler can inline it. There is no virtual call.

```
struct Node {                // one resting order; prev/next are pool
indices
    OrderId      id;
    AgentId      owner;
    Qty          qty;
    std::int32_t prev, next;
    std::int32_t level;      // index into bids_ / asks_
    Side         side;
};
struct Level {               // a FIFO queue threaded through the pool
    std::int32_t head = -1, tail = -1;
```

```
    Qty          total = 0;
};

bool cancel(OrderId id) {
    if (id >= id_map_.size() || id_map_[id] < 0) return false;
    const std::int32_t idx = id_map_[id];
    const Node n = pool_[idx];  // copy: the slot is recycled below
    Level& lvl = (n.side == Side::Buy ? bids_ : asks_)[n.level];

    if (n.prev >= 0) pool_[n.prev].next = n.next; else lvl.head = n.next;  //
unlink
    if (n.next >= 0) pool_[n.next].prev = n.prev; else lvl.tail = n.prev;
    lvl.total -= n.qty;

    if (lvl.head < 0) {
        if (n.side == Side::Buy && n.level == best_bid_) advance_bid();
        if (n.side == Side::Sell && n.level == best_ask_) advance_ask();
    }
    id_map_[id] = -1;
    free_node(idx);
    return true;
}

void advance_bid() { while (best_bid_ >= 0 && bids_[best_bid_].head < 0) --
best_bid_; }
```

C++ points: `template <class Sink> class FastBook` is a class template, so `FastBook<NullSink>` and `FastBook<CountSink>` are two separately compiled classes. `std::int32_t` is a fixed 4-byte integer, which halves the size of a link compared with a pointer. In `cancel`, the node is copied (`const Node n = ...`) before `free_node` recycles its slot, the same capture-before-destroy discipline as in the main book.

Cancelling a node in the middle of a queue is the classic doubly linked list unlink: point the neighbours at each other, and if there was no previous neighbour the level's head moves instead. The `advance_` functions scan toward the next non-empty level when the best level empties. That scan is the flat book's weak spot: on a sparse book with a wide price range it can walk many empty slots. A bitmap with one bit per level and a find-first-set instruction fixes it, but is not implemented here.

Measured results

Each workload replays the identical operation stream through both books, timing only the operations after the book has been built up. Each figure is the best of five repetitions, to reduce the effect of other processes on a shared machine.

Workload	Map and list (ns/op)	Flat (ns/op)	Speed-up
Insert 1,000,000 orders that rest	160	40	4.0×
Cancel 500,000 orders in random order	293	36	8.1×
20,000 marketable orders, each filling about 10 resting orders	3,218	754	4.3×
Mixed: 50% add, 40% cancel, 10% marketable	98	34	2.9×

The table is one run. Across three runs of the benchmark the speed-ups stayed within 3.9 to 4.1× for inserts, 7.7 to 8.3× for cancels, 4.1 to 4.6× for sweeps and 2.9× for the mixed load, but the absolute sweep time for the map-and-list book ranged from 2,260 to 3,218 ns. Read the ratios, not the nanoseconds.

Latency per operation on the mixed workload, in nanoseconds (these include about 29 ns of clock-reading overhead measured separately):

Book	p50	p90	p99	p99.9	Max
Map and list	127	190	300	493	1,512,006
Flat	62	80	130	357	688,476

The typical operation is about twice as fast and the tail is tighter, but both books show a worst case of roughly 0.7 to 1.5 ms. I did not find the cause. On a shared cloud machine an operating-system pause is the likeliest explanation, but a vector growing and copying its contents is another candidate, and the benchmark cannot tell them apart.

Limits of this benchmark. It ran on a shared two-core cloud container, compiled with `g++ 13.3 -O2`, with no CPU pinning or frequency control, so a quiet bare-metal machine would give different absolute numbers. It measures single-threaded replay of synthetic streams with prices clustered near the middle, so it does not capture cache behaviour at production scale. It excludes the agents, the simulator and any network. A flat book that is fast on this test is a good sign and not a guarantee.

Further optimisations (not implemented or measured)

Idea	What it buys	Cost
Bitmap of non-empty levels with a find-first-set instruction	Constant-time best-price search on sparse books	Small code, one more structure to keep in sync
Compiler flags: `-O3`, `-march=native`, link-time optimisation, profile-guided optimisation	Varies by workload; sometimes a visible gain	Binary tied to one CPU model; always re-measure
Pre-fault the memory at start-up (touch the pool and the id map)	Removes page-fault stalls from the hot path	Slower start
Pin each book's thread to a core and isolate that core	Removes scheduler pauses	Operational complexity
A lock-free single-producer, single-consumer ring buffer between gateway and book	Hands orders over without locks	Subtle to get right; use a tested implementation
Cache-line alignment and trimming `Node` to fit fewer lines	Fewer cache misses per operation	Measure with a hardware counter tool such as `perf stat` before and after
Kernel-bypass networking and hardware timestamps	Cuts network-stack latency	Specialist hardware and libraries

The rule that decides which of these to try: measure first, change one thing, run the differential test, measure again. The flat book earned its speed-up by dropping features (no self-trade prevention, fill-or-kill, post-only or modify) and by assuming a bounded price range and ids that the engine hands out. Every one of those is a trade-off a real system must decide on purpose.

Exploits and adversarial behaviour

"Exploit" means three different things around an order book: a flaw in the software that an attacker can abuse, market conduct that abuses the rules of the book itself, and legitimate edges that skilled participants earn. Keeping them apart matters, because the first is fixed with code, the second is policed by regulators and surveillance, and the third is simply how markets work.

Weaknesses in this code

Weakness	How it is abused	Status	Defence
The price band follows the last trade	Two cooperating accounts trade at +9% steps, and each order stays inside the band	**Reproduced**: 10 steps moved the price 2.4×	A slow-moving reference price, volatility halts, surveillance
Market orders ignore the band	One large market order sweeps every level it can reach, which on a thin book means a wild price	Read from the code; the collar fix was checked	Market-to-limit with a collar
No rate limits	Flooding the engine with orders and cancels degrades it for everyone and fills the data feed with noise	Read from the code; the token bucket was checked on its own	Per-participant throttles and order-to-trade ratios
Self-trade prevention keys on `owner`	One real entity running two agent ids can trade with itself and manufacture volume	Read from the code	Put a firm-level identifier on every order and make STP and surveillance use it
Huge or duplicate ids in the flat book	A single order with id 2^{40} crashes the process; a duplicate id leaves an order that cannot be cancelled	**Reproduced**	Engine-assigned ids, duplicate checks at the gateway
The market maker's fair value is the book's mid-price	Anyone who can move the touch moves its quotes	By design; not simulated here	Give the maker an independent price source

The first row deserves the most attention. Every order was individually valid, and the damage came from the pattern. Single-order validation cannot defend against that; only a view across orders and accounts can.

Conduct that real venues police

The behaviours below work, when they work, because an order book is a public signal and a queue. Each is prohibited in many jurisdictions: in the US the Dodd-Frank Act made spoofing an explicit offence under the Commodity Exchange Act, and the EU and UK market-abuse regulations cover the same ground. This is a description for understanding and building defences, not legal advice.

Conduct	The idea	Why it is a problem	How venues look for it
Spoofing and layering	Showing orders with no intention to trade them, to create a false impression of supply or demand	Other participants, and algorithms such as the maker in this tutorial, react to depth that was never real	High cancel rates, large orders cancelled before filling, opposite-side trading while the orders were resting
Quote stuffing	Sending bursts of orders and cancels to slow others or hide activity	Degrades the venue and its data feed	Message-rate and order-to-trade limits
Wash trading	Trading with oneself or a colluding partner	Fake volume and a misleading price	Matching ultimate owners across accounts, self-trade prevention
Momentum ignition	Aggressive trades meant to provoke others into following	Triggers moves unrelated to value	Pattern analysis of price moves around one participant's flow
Marking the close	Trading near the end of the session to move a benchmark price	Distorts settlement prices and index values	Heightened monitoring of closing periods

This product contains the raw material for defence even though it implements none of the attacks. Every event is reported with an order id, owner and timestamp through the `Listener`, so a surveillance process can compute, per owner, the ratio of cancels to trades, the lifetime of cancelled orders and the share of volume against a single counterparty. Writing that process is a good exercise; testing it against a simulated misbehaving agent is how real surveillance teams validate theirs.

Legitimate edges

Edge	Mechanism	Can this simulator test it?
Spread capture	Post both sides and collect the spread from uninformed flow	Yes: the maker's edge was +0.88 ticks per unit
Avoiding adverse selection	Skew and widen quotes so informed traders find them less attractive	Partly: the skew exists, but the maker cannot recognise informed flow
Queue position	Earlier in the FIFO queue means filled first, so a slot at a busy price has real value	Yes: price-time priority is exact, though no agent is built to exploit it
Latency arbitrage	Acting on a price change in one place before slower participants see it	No: market data has no delay and there is one venue
Execution algorithms	Slicing a large order over time to limit impact	Yes, as a new agent, with impact appearing only through the book
Fee and rebate capture	Earning maker rebates	No: fees are not modelled

How venues defend

No single control is enough, so venues layer them: pre-trade checks (size, price band, credit), throttles and kill switches, volatility halts and auctions, self-trade prevention, and post-trade surveillance. Some also change the market design itself. IEX adds a deliberate delay of 350 microseconds to every order to blunt latency advantages, and economists have proposed frequent batch auctions that match once per fixed interval so that speed within the interval earns nothing. Those two ideas matter for the next stage of this project: each can be implemented in the engine in a few dozen lines, and the simulator can then measure what changes.

Other applications of the same machinery

The project contains three reusable ideas: a price-time priority matcher, a deterministic event-driven simulator, and an observer interface that separates logic from reporting. Each carries well beyond equities.

Application	What you reuse	What you must add
Strategy research and backtesting of execution or market making	The book, the agents, the simulator	Calibration to real data, fees, market-data delay, an honest fill model
Training reinforcement-learning agents	A reproducible simulator with a clean `Context` interface; an agent is a function from state to orders	A reward signal, an observation encoder, episode resets, many more realistic counterparties
Rebuilding a live book from an exchange feed	`OrderBook` fed by order-by-order messages (add, delete, execute, replace) instead of orders from agents	A decoder for the feed's binary format and sequence-gap handling
A matching engine for a new venue: crypto, tokens, tickets, carbon credits, prediction markets	The whole core	Accounts, balances and settlement, authentication, persistence, regulation
Intraday power and energy trading	Continuous order books with price-time priority are used for intraday power markets	Contract definitions by delivery period, block and multi-leg orders, grid constraints
In-game economies and auction houses	The book and the engine's risk checks	Item ids instead of a single symbol, fees as a gold sink, anti-fraud rules
Pre-trade impact estimation	Replaying a candidate order against a snapshot of the book to see which levels it would consume	A model of how the book refills after you trade
Surveillance and compliance tooling	The `Listener` event stream	Detection logic per rule, case management, retention
Teaching and interview practice	The code as written	Nothing

Two of the components generalise further.

- **A priority queue with cancellation and FIFO tie-breaking** is exactly what a task scheduler, a timer system, a network packet shaper or a ride-matching queue needs. The sorted map of FIFO lists, plus an id index for cancelling in constant time, is the standard answer to "serve the best item next, break ties by arrival, and let callers withdraw items".
- **The discrete-event kernel** (`at(time, closure)` and a run loop over a priority queue) has nothing market-specific in it. It drives queueing simulations, supply-chain models, network simulators and game servers. Seeded randomness and a `(time, sequence)` ordering are what make any such simulation replayable.

The observer interface is the third reusable idea. It is why the same book served the unit tests (a recorder), the engine (statistics), and the simulator (delivery to agents) without modification. Whenever core logic must feed several consumers that you do not want it to know about, define the events once and let consumers subscribe.

Appendix: source files and further reading

The complete source (1,789 lines in ten files) is delivered as `orderbook.zip` alongside this doc. The code blocks in the tutorial are taken from those files. Everything was compiled with g++ 13.3 in C++17 mode, and the build, tests, simulation, sweep and improvement checks were re-run from a clean unzip to confirm that the numbers in this doc reproduce.

File	Role
`types.hpp`	Scalar types, enums, `Order`, `Trade`, `Listener`
`order_book.hpp`	The book: matching, resting, cancel, modify, validation, invariants
`engine.hpp`	Id allocation, risk checks, ownership checks, statistics
`agents.hpp`	`Context` interface and the four agent types
`sim.hpp`	Event queue, latency, hidden true value, reports and CSV logs
`main.cpp`	Assembles the market; single run or multi-seed sweep
`fast_book.hpp`	Flat-array book for the optimisation section
`tests.cpp`	483 checks including the differential and determinism tests
`bench.cpp`	Benchmark harness for the two books

File	Role
`improve_test.cpp`	Checked sketches for the rate limiter, collar and journal replay

Commands

```
g++ -std=c++17 -O2 -Wall -Wextra -o tests tests.cpp && ./tests
g++ -std=c++17 -O2 -Wall -Wextra -o sim main.cpp
./sim 42                      # one run; writes trades.csv and quotes.csv
./sim sweep 100               # 100 seeds, default maker settings
./sim sweep 100 1000 5        # maker latency 1000 us, refresh 5 ms
g++ -std=c++17 -O2 -Wall -Wextra -o bench bench.cpp && ./bench
g++ -std=c++17 -O2 -fsanitize=address,undefined -g -o tests_asan tests.cpp &&
./tests_asan
```

Exercises

1. Add a stop-order book and test it with the differential method against a brute-force implementation.
2. Add market-data latency to `Context::quote()` and rerun the maker-speed experiment. Does speed start to matter?
3. Give the market maker an independent price signal and compare its edge with and without it.
4. Write a surveillance `Listener` that flags owners with a high cancel-to-trade ratio, then build a misbehaving agent to test it.
5. Implement a call auction and use it for a volatility halt.
6. Replace the flat book's id array with a hash map, and rerun the differential test and the benchmark.

Further reading

These are titles from memory, offered as starting points. I did not open them while writing this tutorial, and none of this tutorial's numbers depend on them.

- Maureen O'Hara, *Market Microstructure Theory*: the economics of spreads, information and market makers.
- Bouchaud, Bonart, Donier and Gould, *Trades, Quotes and Prices*: empirical facts about order books and price impact.
- Cartea, Jaimungal and Penalva, *Algorithmic and High-Frequency Trading*: market-making and execution models.

- Avellaneda and Stoikov, "High-frequency trading in a limit order book" (2008): the reservation-price market-making model.
- Glosten and Milgrom (1985) and Kyle (1985): the classic models of informed trading and adverse selection.
- Budish, Cramton and Shim, "The High-Frequency Trading Arms Race: Frequent Batch Auctions as a Market Design Response" (2015).
- Byrd, Hybinette and Balch, "ABIDES: Towards High-Fidelity Multi-Agent Market Simulation": an open agent-based market simulator.
- Nasdaq's TotalView-ITCH specification: a real order-by-order feed format to practise rebuilding a book from.
- Scott Meyers, *Effective Modern C++*, and the reference site cppreference.com for the language itself.